

4. Deadlock

1. Principles of Deadlock
2. Resource Status Modeling Conditions For Deadlock
3. Methods For Handling Deadlocks
 - a. Prevention
 - b. Avoidance
 - c. Detection
 - d. Recovery

4. Deadlock

4.1 Definition

In a multiprogramming system, processes request resources. If those resources are being used by other processes then the process enters a waiting state. However, if other processes are also in a waiting state, we have deadlock.

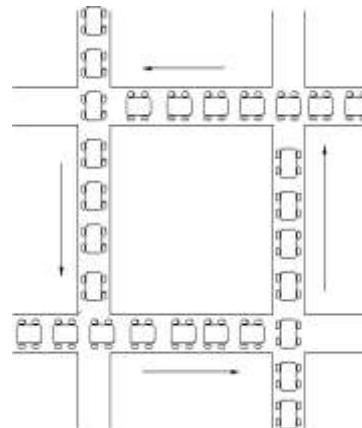
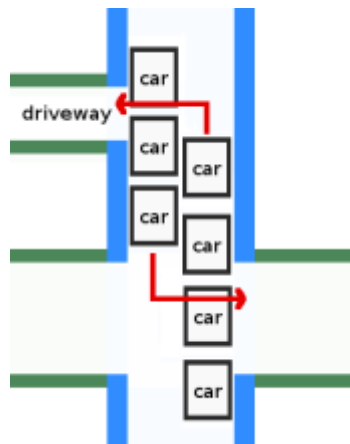
Definition: A set of processes is in a deadlock state if every process in the set is waiting for an event (release) that can only be caused by some other process in the same set.

System Model

Each process utilizes a resource as follows:

- request
- use
- release

1. Principles of Deadlock



Example 4.1

Process-1 requests the printer, gets it
Process-2 requests the tape unit, gets it

Process-1 and

Process-1 requests the tape unit, waits
Process-2 requests the printer, waits

Process-2 are
deadlocked!

Understanding the principles of deadlock in designing systems that can avoid or handle this situation. The key principles include:

1. **Mutual Exclusion:** At least one resource must be held in a non-shareable mode. Only one process can use the resource at any given instant. If another process requests that resource, the requesting process must be delayed until the resource is released.
2. **Hold and Wait:** A process holding at least one resource is waiting to acquire additional resources that are currently being held by other processes.
3. **No Preemption:** A resource can only be released voluntarily by the process holding it, after that process has completed its task.
4. **Circular Wait:** A set of processes are waiting such that each process is waiting for a resource that is held by the next process in the set, forming a circular chain.

Resource Status Modeling Conditions For Deadlock

Let's explain all four conditions related to deadlock in the context of the scenario with two processes and two resources:

Deadlock is a condition in a multiprogramming environment where two or more processes are unable to proceed because each is waiting for one of the others to release a resource.

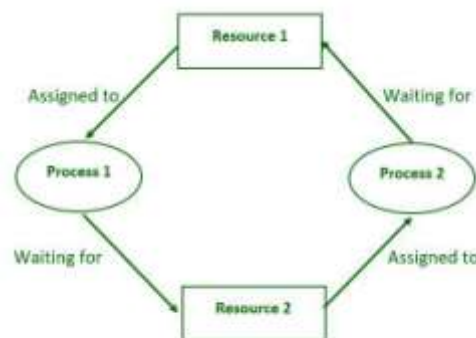


Figure: Deadlock in Operating system

Mutual Exclusion

This condition requires that at least one resource be held in a non-shareable mode, which means that only one process can use the resource at any given time. Both Resource 1 and Resource 2 are non-shareable in our scenario, and only one process can have exclusive access to each resource at any given time.

As an example:

- Process A obtains Resource 1.
- Process B acquires Resource 2.

Hold and Wait

The hold and wait condition specifies that a process must be holding at least one resource while waiting for other processes to release resources that are currently held by other processes. In our example,

- Process A has Resource 1 and is awaiting Resource 2.
- Process B currently has Resource 2 and is awaiting Resource 1.

- Both processes hold one resource while waiting for the other, satisfying the hold and wait condition.

No Preemption

Preemption is the act of taking a resource from a process before it has finished its task. According to the no preemption condition, resources cannot be taken forcibly from a process can only release resources voluntarily after completing its task.

In our scenario, neither Process A nor Process B can be forced to release the resources in their possession. The processes can only release resources voluntarily.

Circular Wait

This condition implies that circular processes must exist, with each process waiting for a resource held by the next process in the chain.

In our scenario, Process A is waiting for Resource 2, which is being held by Process B. Process B is awaiting Resource 1 from Process A.

This circular chain of dependencies causes a deadlock because neither process can proceed, resulting in a system shutdown.

To summarise, in the context of two processes and two resources, all four conditions (mutual exclusion, hold and wait, no preemption, and circular wait) must be met at the same time. Deadlocks are a major concern in operating systems, and various techniques are used to avoid them to prevent, detect, and recover from them.

Methods For Handling Deadlocks

- a. Prevention
- b. Avoidance
- c. Detection
- d. Recovery



Methods for Handling Deadlocks

- Ensure that the system will *never* enter a deadlock state
- Allow the system to *enter* a deadlock state and then *recover*
- *Ignore* the problem and pretend that deadlocks never occur in the system; used by most operating systems, including UNIX



Deadlock Prevention

Restrain the ways request can be made

- **Mutual Exclusion** – not required for sharable resources (i.e. a printer); must hold for nonsharable resources (i.e. read-only files)
- **Hold and Wait** – must guarantee that whenever a process requests a resource, it does not hold any other resources
 - Require process to request and be allocated all its resources before it begins execution, or allow process to request resources only when the process has none
 - Low resource utilization; starvation possible



Deadlock Prevention (Cont.)

■ No Preemption –

- If a process that is holding some resources requests another resource that **cannot be immediately allocated** to it, then all resources currently being held are released
- Preempted resources are added to the list of resources for which the process is waiting
- Process will be restarted only when it can regain its old resources, as well as the new ones that it is requesting

■ Circular Wait – impose a total ordering of all resource types, and require that each process requests resources in an increasing order of enumeration



Deadlock Avoidance

Requires that the system has some additional on how resources are to be requested.

- Simplest and most useful model requires that each process declare the *maximum number* of resources of each type that it may need
- The deadlock-avoidance algorithm dynamically examines the resource-allocation state to ensure that there can never be a circular-wait condition
- Resource-allocation *state* is defined by the number of available and allocated resources, and the maximum demands of the processes



Avoidance algorithms

- **Single instance** of a resource type
 - Use a resource-allocation graph
- **Multiple instances** of a resource type
 - Use the banker's algorithm



Deadlock Detection

- Allow system to enter deadlock state
- Detection algorithm
- Recovery scheme

ALGORITHM:**Step-1:** Start the program.**Step-2:** Declare the memory for the process.**Step-3:** Read the number of process, resources, allocation matrix and available matrix.**Step -4:** Calculate the need matrix : $\text{need} = \text{max} - \text{allocation}$ **Step-4:** Compare each and every process using the banker's algorithm.**Step-5:** If the process is in safe state then it is a not a deadlock process otherwise it is a deadlock process**Step-6:** produce the result of state of process**Step-7:** Stop the program

$m=3, n=5$ Step 1 of Safety Algo

Work = Available

Work =

3	3	2
---	---	---

0 1 2 3 4

Finish =

false	false	false	false	false
-------	-------	-------	-------	-------

For $i=0$ Step 2

Need₀ = 7, 4, 3 ✗

Finish [0] is false and Need₀ > Work

So P₀ must wait But Need ≤ Work

For $i=1$ Step 2

Need₁ = 1, 2, 2 ✓

Finish [1] is false and Need₁ < Work

So P₁ must be kept in safe sequence

Step 3

Work = Work + Allocation₁

Work =

5	3	2
---	---	---

0 1 2 3 4

Finish =

false	true	false	false	false
-------	------	-------	-------	-------

For $i=2$ Step 2

Need₂ = 6, 0, 0 ✗

Finish [2] is false and Need₂ > Work

So P₂ must wait

For $i=3$ Step 2

Need₃ = 0, 1, 1 ✓

Finish [3] = false and Need₃ < Work

So P₃ must be kept in safe sequence

Step 3

Work = Work + Allocation₃

Work =

7	4	3
---	---	---

0 1 2 3 4

Finish =

false	true	false	true	false
-------	------	-------	------	-------

For $i=4$ Step 2

Need₄ = 4, 3, 1 ✓

Finish [4] = false and Need₄ < Work

So P₄ must be kept in safe sequence

Step 3

Work = Work + Allocation₄

Work =

7	4	5
---	---	---

0 1 2 3 4

Finish =

false	true	false	true	true
-------	------	-------	------	------

For $i=0$ Step 2

Need₀ = 7, 4, 3 ✓

Finish [0] is false and Need₀ < Work

So P₀ must be kept in safe sequence

Step 3

Work = Work + Allocation₀

Work =

7	5	5
---	---	---

0 1 2 3 4

Finish =

true	true	false	true	true
------	------	-------	------	------

For $i=2$ Step 2

Need₂ = 6, 0, 0 ✓

Finish [2] is false and Need₂ < Work

So P₂ must be kept in safe sequence

Step 3

Work = Work + Allocation₂

Work =

10	5	7
----	---	---

0 1 2 3 4

Finish =

true	true	true	true	true
------	------	------	------	------

Step 4

Finish [i] = true for $0 \leq i \leq n$

Hence the system is in Safe state

The safe sequence is P₁, P₃, P₄, P₀, P₂

Considering the following example of a system, check whether the system is safe or not, using Banker's algorithm. Determine the sequence if it is safe.

Process	Allocation	Max.	(work)	(max-allocation)
			Available	Need
	A B C	A B C	A B C	A B C
P ₀	0 1 0	7 5 3	3 3 2	7 4 3
P ₁	2 0 0	3 2 2	5 3 2	1 2 2
P ₂	3 0 2	9 0 2	7 4 3	6 0 0
P ₃	2 1 1	2 2 2	7 4 5	0 1 1
P ₄	0 0 2	4 3 3	7 5 5	4 3 1
			10 5 7	

< P₁, P₃, P₄, P₀, P₂ >

Banker's algorithm:

$Need_i \leq work \Rightarrow work = work + allocation.$

$$P_0 \quad 743 \leq 332 \quad \times$$

$$P_1 \quad 122 \leq 332 \quad \checkmark \quad \begin{aligned} W &= W + alloc \\ &= 332 + 200 \\ &= 532 \end{aligned}$$

$$P_2 \quad 600 \leq 532 \quad \times$$

$$P_3 \quad 011 \leq 532 \quad \checkmark \quad \begin{aligned} W &= W + alloc \\ &= 532 + 211 \\ &= 743 \end{aligned}$$

$$P_4 \quad 431 \leq 532 \quad \checkmark \quad \begin{aligned} W &= W + alloc \\ &= 743 + 002 \\ &= 745 \end{aligned}$$

$$P_0 \quad 743 \leq 745 \quad \checkmark \quad \begin{aligned} W &= W + alloc \\ &= 745 + 010 \\ &= 755 \end{aligned}$$

$$P_2 \quad 600 \leq 755 \quad \checkmark \quad \begin{aligned} W &= W + alloc \\ &= 755 + 302 \\ &= 1057 \end{aligned}$$



Resource-Allocation Graph

A set of vertices V and a set of edges E .

- V is partitioned into two types:
 - $P = \{P_1, P_2, \dots, P_n\}$, the set consisting of all the processes in the system
 - $R = \{R_1, R_2, \dots, R_m\}$, the set consisting of all resource types in the system
- request edge – directed edge $P_i \rightarrow R_j$
- assignment edge – directed edge $R_j \rightarrow P_i$



Resource-Allocation Graph (Cont.)

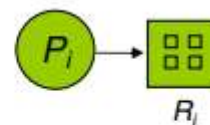
- Process



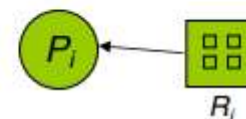
- Resource Type with 4 instances



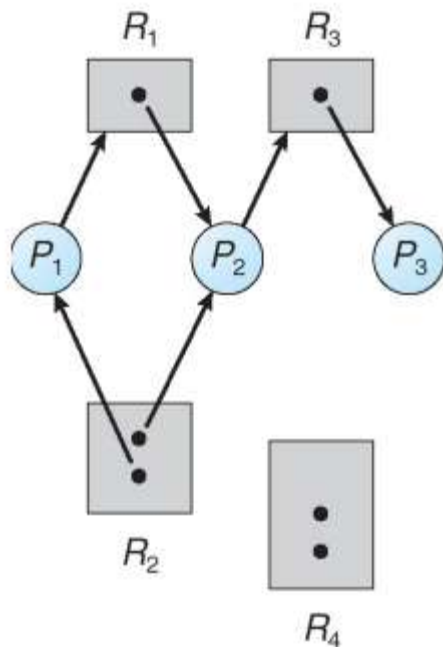
- P_i requests instance of R_j



- P_i is holding an instance of R_j



Example of a Resource Allocation Graph



$P = \{P_1, P_2, P_3\}$
 $R = \{R_1, R_2, R_3, R_4\}$
 $E = \{P_1 \rightarrow R_1, P_2 \rightarrow R_3, R_1 \rightarrow P_2, R_2 \rightarrow P_2, R_2 \rightarrow P_1, R_3 \rightarrow P_3\}$

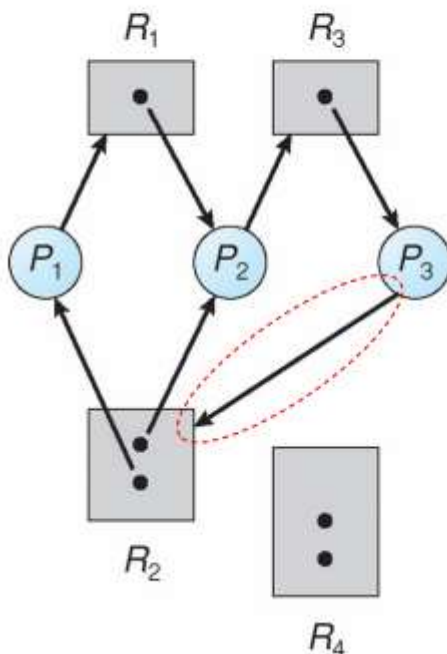
Resource instances:

- One instance of resource type R_1 ,
- Two instances of resource type R_2 ,
- One instance of resource type R_3 ,
- Two instances of resource type R_4

Process states

- P_1 is holding an instance of R_2 and waiting for an R_1
- P_2 is holding an R_1 and an R_2 and is waiting for an R_3
- P_3 is holding an R_3

Resource Allocation Graph With A Deadlock



Suppose P_3 requests an instance of resource type R_2 .

Two cycles exist in the system:

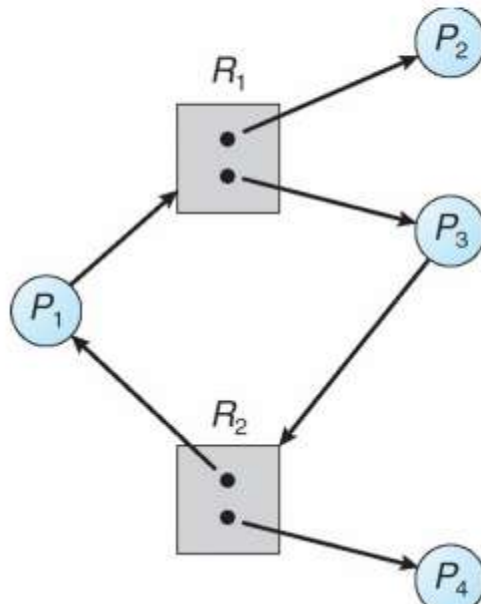
$P_1 \rightarrow R_1 \rightarrow P_2 \rightarrow R_3 \rightarrow P_3 \rightarrow R_2 \rightarrow P_1$

$P_2 \rightarrow R_3 \rightarrow P_3 \rightarrow R_2 \rightarrow P_2$

Processes P_1 , P_2 , P_3 are deadlocked.



Graph With A Cycle But No Deadlock



We have a cycle but no deadlock.

Process P4 may release its instance of resource type R2. R2 can then be allocated to P3, breaking the cycle.

Strategies to Handle Deadlock

1. **Deadlock Prevention:** Ensure that at least one of the necessary conditions for deadlock cannot hold.
 - **Mutual Exclusion:** This condition is difficult to deny because some resources are inherently non-shareable.
 - **Hold and Wait:** Require processes to request all required resources at once, or ensure a process cannot hold one resource while requesting another.
 - **No Preemption:** Allow preemption where possible, i.e., if a process holding some resources is requested to release them, it should release them.
 - **Circular Wait:** Impose a total ordering of all resource types and require that each process requests resources in an increasing order of enumeration.
2. **Deadlock Avoidance:** Dynamically examine the resource-allocation state to ensure that a circular wait condition can never exist. The system must have prior knowledge of the maximum resource requirement of each process. Algorithms like Banker's Algorithm can be used.
3. **Deadlock Detection and Recovery:** Allow the system to enter a deadlock state, detect it, and then recover.
 - **Detection:** Implement an algorithm that checks for cycles in the resource allocation graph.
 - **Recovery:** Once a deadlock is detected, take action to recover. This can involve terminating processes involved in the deadlock or preempting some resources from the deadlocked processes.
4. **Deadlock Ignorance:** In some systems, especially where deadlocks are very rare and the cost of prevention or detection is high, the strategy might be to ignore the problem and restart the system if a deadlock occurs. This is used in many real-world systems.

Deadlock Examples

Dining Philosophers Problem

In this classic synchronization problem, five philosophers sit at a table with five chopsticks. Each philosopher needs two chopsticks to eat but only one chopstick is available between each pair. If each philosopher picks up the chopstick to their right at the same time, none of them can eat, leading to a deadlock.

Resource Allocation Graph

A visual and analytical way to understand deadlock is through resource allocation graphs, where nodes represent processes and resources, and edges represent the current allocation of resources to processes or processes waiting for resources. Cycles in this graph indicate potential deadlocks.

Understanding and addressing deadlock requires careful design and implementation of resource allocation and process management strategies in operating systems and concurrent programming environments.

Frequently Asked Questions on Deadlock

Q.1: What are the necessary conditions for a deadlock to occur?

Answers:

Deadlock can occur if four conditions are present simultaneously:

Mutual Exclusion: At least one resource must be held in a non-sharable mode, meaning it cannot be simultaneously used by multiple processes.

Hold and Wait: A process must be holding at least one resource while waiting for another resource to be released by another process.

No Preemption: Resources cannot be forcibly taken away from a process. They must be released voluntarily.

Circular Wait: A circular chain of two or more processes exists, where each process is waiting for a resource held by another process in the chain.

Q.2: Can a deadlock occur if only one of the necessary conditions is present?

Answers:

No, a deadlock cannot occur if only one of the necessary conditions is present. All four conditions must be satisfied simultaneously for a deadlock to occur.

Q.3: Can a single-processor system experience a deadlock?

Answer:

Yes, both single-processor and multi-processor systems can experience deadlock.

Deadlocks are not dependent on the number of processors, but rather on the previously mentioned conditions.

Q.4: Can a deadlock occur if only one of the required conditions exists?

Answer:

No, a deadlock cannot occur if only one of the required conditions is met. For a deadlock to occur, all four conditions must be met at the same time.

Q.5: Can a single-processor system experience a deadlock?

Answer:

Yes, both single-processor and multi-processor systems can experience deadlock. Deadlocks are not dependent on the number of processors, but rather on the previously mentioned conditions.

https://eng.libretexts.org/Courses/Delta_College/Operating_System%3A_The_Basics/06%3A_Deadlock/6.1%3A_Concept_and_Principles_of_Deadlock

<https://education.sakshi.com/en/cseit/study-material/operating-system/principles-deadlock-44708>

<https://users.metu.edu.tr/halici/courses/442/Ch5%20Deadlocks.pdf>

<https://www.utc.edu/sites/default/files/2021-04/2800-lecture7-deadlocks.pdf>

<https://www.slideshare.net/slideshow/osv-unit-5-deadlock/251549474>

<https://slideplayer.com/slide/6842128/>

<https://redirect.cs.umbc.edu/courses/graduate/621/fall02/lectures/ch3.pdf>

Banker Alg;

https://www.youtube.com/watch?v=2V2FfP_olaA&si=9-7feIgl1dE45bKO

<https://www.youtube.com/watch?v=5cBOr-ijXIM>